Email authentication end to end: SPF, DKIM and DMARC

> **Belona Cybersecurity** · October 2026 Real case on our own domain: `belonacyber.com`, with Google Workspace.
> *Versión en español:* `../autenticacion-de-correo-spf-dkim-dmarc.md`

The problem it solves

Email was designed without authentication. The **"From:"** a recipient sees **is text the sender typed** — exactly like the return address on a paper envelope: nobody at the post office checks whether it's true.

That's why anyone can send mail that appears to come from a company. For an office handling financial data — a tax preparer, an accountant, a law firm — that isn't an annoyance: **it's the most common fraud path against their clients and against the IRS.**

The three controls close that door, and each answers a different question:

	Question it answers	Analogy
SPF	did it leave an authorized server?	the list of couriers allowed to ship in my name
DKIM	is it from who it claims, and intact?	the seal inside the letter
DMARC	what do I do when one fails?	the instruction to the courier, plus a report

All three are necessary and none replaces another. SPF doesn't detect modification in transit; DKIM doesn't say where it came from; and without DMARC **neither one is enforced** — the receiving server has no instruction for what to do when they fail.

The most common mistake: `p=none` with no `rua`

The most frequent DMARC record in the world is this one:

```
v=DMARC1; p=none
```

And it **does nothing at all.**

- `p=none` means *"don't block anything"*
- with no `rua=`, it doesn't collect reports either

`p=none` is supposed to be the **monitoring** phase: you don't block yet, but you **watch** who sends mail in your name. With no report address, it watches nothing. It's a record that exists so someone could tick a box.

Never say "they have no DMARC" when the record exists. The precise statement is: *it's published, but in monitoring mode and with no report address, so it isn't even monitoring*. That precision is the difference between sounding like someone who read the configuration and someone who ran an online tool.

Staged rollout

The mistake that takes down a company's email is setting `p=reject` on day one. If some legitimate system sends on behalf of the domain and nobody remembered it — an invoicing tool, a form, a shipping service — its mail starts bouncing **and nobody finds out until a client complains.**

The correct sequence:

Stage	Record	What happens	How long
1 · Monitor	`p=none; rua=...`	nothing blocked; daily reports arrive	2–4 weeks
2 · Quarantine	`p=quarantine; rua=...`	failures go to spam, not lost	4–8 weeks
3 · Reject	`p=reject; rua=...`	failures aren't delivered	permanent

Every stage is reversible and each one needs evidence before the next. You don't advance on a calendar: you advance when the reports show everything legitimate is passing.

In this case: `p=quarantine` active since October, moving to `p=reject` in December **after a month of clean reports**.

Reading the first report

DMARC reports arrive as compressed XML, one per provider per day. This is the first one received, in full:

```
<policy_published>
  <domain>belonacyber.com</domain>
  <p>quarantine</p>
  <pct>100</pct>
</policy_published>
<record>
  <row>
    <source_ip>209.85.220.41</source_ip>
    <count>1</count>
    <policy_evaluated><dkim>pass</dkim><spf>pass</spf></policy_evaluated>
  </row>
  <auth_results>
```

```
    <dkim><domain>belonacyber.com</domain><result>pass</result>
          <selector>google</selector></dkim>
    <spf><domain>belonacyber.com</domain><result>pass</result></spf>
  </auth_results>
</record>
```

What it says, in order of importance:

1. **The published policy is the one you think it is.** `p=quarantine` — the world sees it.
2. `source_ip` **209.85.220.41** resolves to `mail-sor-f41.google.com`: our own server. **Nobody else is sending in the domain's name.**
3. **SPF and DKIM both** `pass`, with selector `google` — which **confirms DKIM is configured and signing**, something you cannot determine from outside without knowing the selector.
4. `disposition: none` — nothing was blocked because nothing needed to be.

The report isn't only for spotting attacks. It's how you learn what you can safely block — which is what authorizes the move to `p=reject`.

The DNS migration trap

During this work the domain moved from its registrar to a different DNS provider. That's the operation where most mail gets lost, for one specific reason:

The DKIM record is 408 characters. A TXT record can't exceed 255 per string, so it is split internally in two. **That's where it gets truncated when copied between providers** — and the failure is silent:

- mail **keeps going out**
- the signature stops validating
- it starts landing in spam
- and it surfaces only in the next DMARC report, as `dkim=fail`

How it was avoided

1. Capture the prior state before touching anything, as a document, including the original nameservers so the change can be reverted.

2. Check DNSSEC. If it's active and the nameservers change, the domain **stops resolving entirely** — mail included — because the signatures no longer match what the `.com` registry has on record. Verified: no `DS` record, safe to migrate.

3. Compare the DKIM character by character, not by eye:

```
new provider : 408 characters
live         : 408 characters
```

```
>>> IDENTICAL <<<
ends in IDAQAB: True
```

`IDAQAB` is the closing sequence of an RSA public key. If it ends in anything else, it was truncated.

4. Verify the four critical records afterwards, don't assume them:

```
MX     smtp.google.com (pri 1)             OK
SPF    single record, intact               OK
DKIM   408 characters, ends in IDAQAB      OK
DMARC  p=quarantine                        OK
```

Zero mail downtime.

Coexisting with a second provider

A transactional sending service was later added for the site's contact form. That's where **the mistake that breaks a company's SPF** appears:

> **A domain can only have ONE SPF record.**

If `v=spf1 include:_spf.google.com ~all` already exists and a second `v=spf1 include:other-provider ~all` is added, **receiving servers see two records, can't tell which applies, and both fail.** The entire authentication is lost by adding a provider.

Two correct ways out:

- **Merge into a single line**: `v=spf1 include:_spf.google.com include:other ~all`
- **Or have the provider use a delegated subdomain** (`send.domain.com` via CNAME) carrying its own SPF. Then there's no conflict, because they don't share a name.

Here the provider used the second. The verified result:

```
YOUR MAIL                    NEW PROVIDER
MX    Google           OK    DKIM   own selector     OK
SPF   single record    OK    CNAME  send             OK
DKIM  selector google  OK    CNAME  rsend            OK
DMARC p=quarantine     OK
```

Two mail systems signing for the same domain, with different DKIM selectors, without touching each other.

What this does NOT cover

- **DMARC doesn't protect the display name.** An attacker can set *"Belona Cybersecurity"* as the display name while sending from `gmail.com`, and DMARC says nothing: the From domain is `gmail.com`, and that one is authenticated. Staff training handles that, not DNS.
- **It doesn't protect lookalike domains.** `belonacyberr.com` is a different domain with its own DMARC. That's handled by monitoring similar registrations.
- `~all` **(softfail) vs** `-all` **(hardfail):** with DMARC at enforcement the difference is small, but `-all` is more explicit. Pending at the move to `p=reject`.
- **BIMI** — making the logo appear beside the message — requires DMARC at enforcement **plus a Verified Mark Certificate** costing over a thousand dollars a year. Out of scope for now.

What a client receives

1. The three records configured and **verified from two independent public resolvers**, not just from the provider's panel.
2. The rollout **in stages**, with the date of the next one and the criterion for advancing.
3. **A monthly report in plain language**: who tried to send mail in their name, what happened, and whether anything legitimate of theirs is failing.
4. Documentation of what changed and how to revert it.

Point 3 is what changes the conversation. **Without** `rua=` **, the business has a policy and no visibility.** With it, every month they know who is trying to impersonate them — and that turns a technical chore into a control you can actually show someone.

From the technical portfolio of Belona Cybersecurity. Performed on our own domain; the values shown are real.
